

Verteilte Systeme Abgabe 4

1)

1.1 Anforderungen an das System:

Funktional:

Status von Flügen einsehen
Neue Flugdaten empfangen
Anfragen zu spezifischen Flügen bearbeiten
Reservierungen tätigen
Keine Preise nachträglich anheben, senken aber schon

Nicht-Funktional:

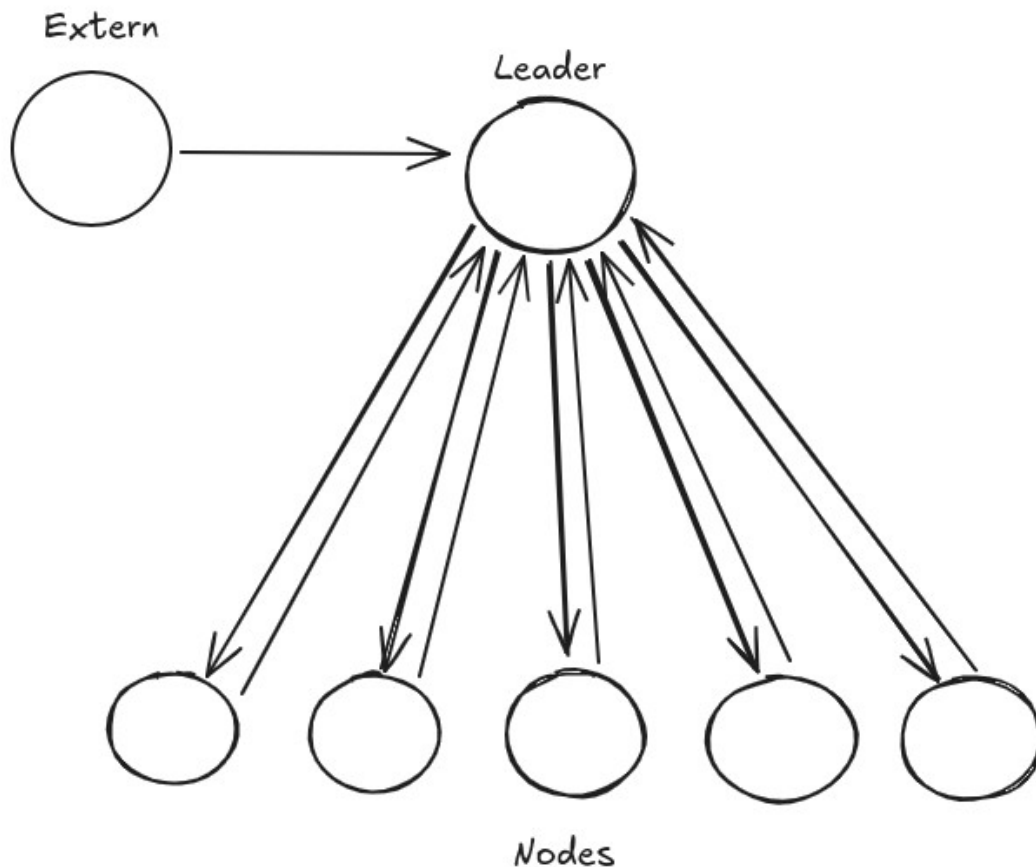
Mehrere Nodes/Server
Muss Burst-Traffic standhalten
Verarbeitung innerhalb von Sekunden
Skalierbar
Performant
Fehlertolerant
Hunderttausende Reisende gleichzeitig online

1.2

Ich benutze eine Architektur mehreren Nodes, die die Regionen zum Beispiel nach Kontinent aufteilen und jeweils für diese Gebiete die Anfragen bearbeiten. Da die meisten Flüge innerhalb der Kontinente stattfinden, können diese zu einem gewissen Teil autonom agieren. Das heißt auch, dass die meisten Benutzer einen Geographisch nahen Server ansprechen, welcher eine geringere Latenz bietet und viele Benutzer gleichzeitig versorgen kann.

Flüge über mehrere Gebiete hinaus werden in dem Startgebiet gespeichert. Flugdaten von anderen Gebieten werden dann über den Leader angefragt. Dieser verwaltet die Clock mit Lamport Timestamps, da jede Übertragung durch ihn erfolgt. Die externen Daten, die jede Stunde von Partnerfirmen ankommen, werden an den Leader gesendet, welcher diese dann verteilt.

Dieses System ist außerdem komplett gegen Partitionen tolerant, welches natürlich nach dem CAP-Theorem wieder andere Nachteile mit sich zieht. Zum Beispiel ist zumindest der Internationale Flugverkehr nicht erreichbar, solange der Leader nicht erreichbar ist. Da es sich hierbei jedoch nicht um die Daten zum Lotzen der Flugzeuge, sondern lediglich um Buchungen und Flugauskünfte handelt, ist hier nicht 99,9999% Erreichbarkeit nötig, weshalb hier Abstriche gemacht werden können. Die Konsistenz hingegen, kann durch die Replizierung in den einzelnen Nodes sowie durch die Bestätigung von Nachrichten gewährleistet werden. Die



2)

Einschätzung:

Der Prototyp erfüllt seine Aufgabe, Nachrichten zwischen mehreren Clients auszutauschen. Die Architektur erlaubt theoretisch so viele Benutzer, wie es Ids gibt (als int gespeichert). Dabei verwendet der Server Heartbeat-Nachrichten, um festzustellen, ob ein Client die Verbindung verloren hat und teilt das Usern mit, wenn sie versuchen, den Client zu benachrichtigen. UserIds werden an Adressen gebunden, weshalb es nicht möglich ist mehreren Usern die selbe Id zu geben oder eine vorher benutzte Id eines nun inaktiven Benutzers wiederzuverwenden.

Der Chatverlauf wird in einem Browserfenster angezeigt, während die App in der Konsole gestartet werden muss und dort die Nachrichten geschrieben werden müssen. Das Schreiben von Nachrichten im Browser ist leider nicht möglich.

Es gibt keine Speicherung der Nachrichten, weshalb es nicht möglich ist, vorherige Nachrichten nach einem Neuverbinden zu sehen, oder Nachrichten zu lesen, die man bekommen hat, während man offline war.

Da es sich um einen Server mit mehreren Clients handelt, ist hier Partitionstoleranz gegeben, da die Clients nicht ohne Server funktionieren können. Das geht jedoch auf Kosten der Erreichbarkeit. Die Konsistenz der Daten wird in diesem Prototypen auch nicht sichergestellt, da Daten weder Paritätschecks unterlaufen noch mehrfach gespeichert oder ausgerechnet werden.

Außerdem gibt es keinerlei Zeitsynchronisierung, weshalb es dazu kommen kann, dass zuerst gesendete Daten zuletzt ankommen und als zuletzt gesendet angesehen werden.

Außerdem wurde, bei diesem Prototypen nicht auf Sicherheit geachtet, weshalb die Verbindung auf demselben Gerät mit derselben UserId möglich ist, ohne ein Passwort einzugeben. Da die UserIds mithilfe von Adresse und Port vergeben werden und die Ports zufällig gewählt werden bei der Verbindung, ist es jedoch nicht möglich, garantiert denselben User nach dem erneuten Verbinden zu erhalten.